

NIMRAG

SOFTWARE ENGINEERING · 2026

Nimrag.

Intelligenter Spiegel mit
berührungsloser
Gestensteuerung

Louis Reißer

Backend · Gestensteuerung

Sebastian Rieger

Frontend · Widgets

Jannik Haase

Hardware · Spotify-Widget

Agenda

01 **Produktvision**

02 **User Experience**

Das Widget- und Gridsystem · Gestensteuerung

03 **Technische Umsetzung**

Systemarchitektur & Design · Backend & Gestensubsystem · Frontend-Architektur

04 **Hardware**

05 **Qualität & CI/CD**

06 **Projektmanagement**

07 **Ausblick & Live-Demo**

1 • Produktvision — Was ist Nimrag?

Das Projekt

Nimrag ist ein Smart Mirror: ein halbdurchlässiger Spiegel mit dahinterliegendem Display, der Informationen als Overlay darstellt.

Kernfunktionen:

- Anzeige von Alltagsinformationen (Wetter, Uhrzeit, Nachrichten, Warnungen)
- Berührungslose Steuerung über Kamera-basierte Gestensteuerung
- Modulares Widget-System für flexible Inhalte

Technischer Ansatz:

- Eigenentwicklung: Frontend (Vue.js) + Backend (FastAPI)
- Hardware: Raspberry Pi · Kamera · Zwei-Wege-Spiegel
- Automatisierte Build- und Deployment-Pipeline

Projektziel

Entwicklung eines funktionsfähigen Prototyps mit:

- Konfigurierbarem Widget-Grid
- Zuverlässiger berührungsloser Gestensteuerung
- Getesteter und dokumentierter Codebasis
- Deployment auf dedizierter Hardware

02

User Experience

Das Widget- und Gridsystem &
Gestensteuerung

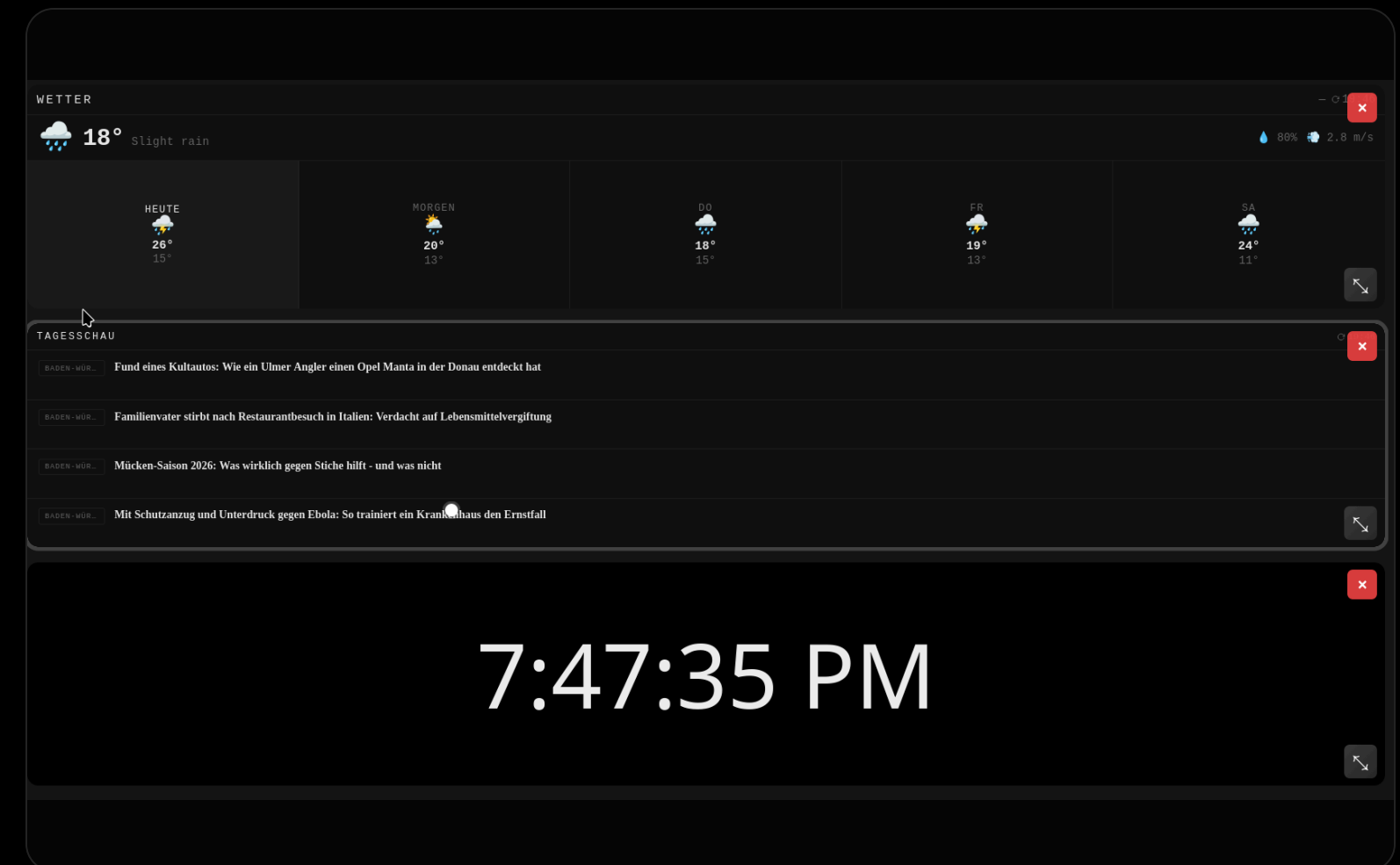
2.1 · Das Widget- und Gridsystem

Das Grid

- Widgets frei auf dem Grid positionierbar
- Mehrere Widgets gleichzeitig sichtbar
- Widgets verschiebbar, skalierbar, löschtbar
- Änderungen werden persistiert

Verfügbare Widgets

- Wetter · Uhrzeit · Nachrichten (Tagesschau)
- NINA-Katastrophenwarnungen · Börsenkurse
- Kamera-Vorschau · Frage des Tages
- Useless Fact · Corporate Bullshit of the Hour
- Zufalls-Meme · Animiertes GIF



Live-Ansicht — aktiver Cursor auf dem Widget-Grid

2.2

Gestensteuerung

Berührungslose Bedienung für den Alltag

Warum keine Maus — warum Gesten?

Das Problem:

- Wandmontiert → keine Tastatur oder Maus in Reichweite
- Kontext: Badezimmer, Flur, Küche → Hygiene, nasse Hände

Die Lösung:

- Kamera erkennt Handbewegungen vor dem Spiegel
- Komplette **berührungslos** aus 50–80 cm Abstand
- Keine Hardware-Eingabegeräte notwendig
- Ergänzung: Sprachsteuerung (*in Entwicklung*)

Gestensteuerung ist kein Add-on — sie ist die einzig sinnvolle Eingabemethode für einen wandmontierten Spiegel.

Die vier Eingabemethoden

Wischen (*Swipe*)

Offene Handfläche horizontal Wischen
→ Navigiert zum nächsten / vorherigen Widget

Kreis

Kreisbewegung beschreiben, Hand am Ende öffnen
→ Bestätigt eine Aktion / öffnet Menü

Cursor + Halten

Zeigefinger bewegt den Cursor — 1 Sekunde auf einem Button halten
→ Löst Klick aus (Löschen, Größe ändern, Menü)

Pinch-to-Move

Daumen + Zeigefinger zusammen = Widget greifen
Hand bewegen = Widget folgt, Finger öffnen = ablegen
→ Drag & Drop für das Widget-Grid

Frühere Ansätze wie Push-Klick-Gesten oder Zwei-Hand-Zoom wurden verworfen — Cursor-Hold löst dasselbe eleganter.

Gesten nach Kontext

Kamera-Vorschau (*Startbildschirm*)

Beim Start zeigt die App die Kamera-Vorschau im Vollbild.

 Wischen nach links — zum Widget-Grid wechseln

Widget-Grid (Normalmodus)

 Kreis — Edit-Modus aktivieren

 Cursor halten — Buttons & Menüs klicken

Edit-Modus

 Pinch — Widget greifen & verschieben

 Pinch auf leerem Feld — Widget-Shop öffnen

 Cursor halten — Widget löschen · Größe ändern

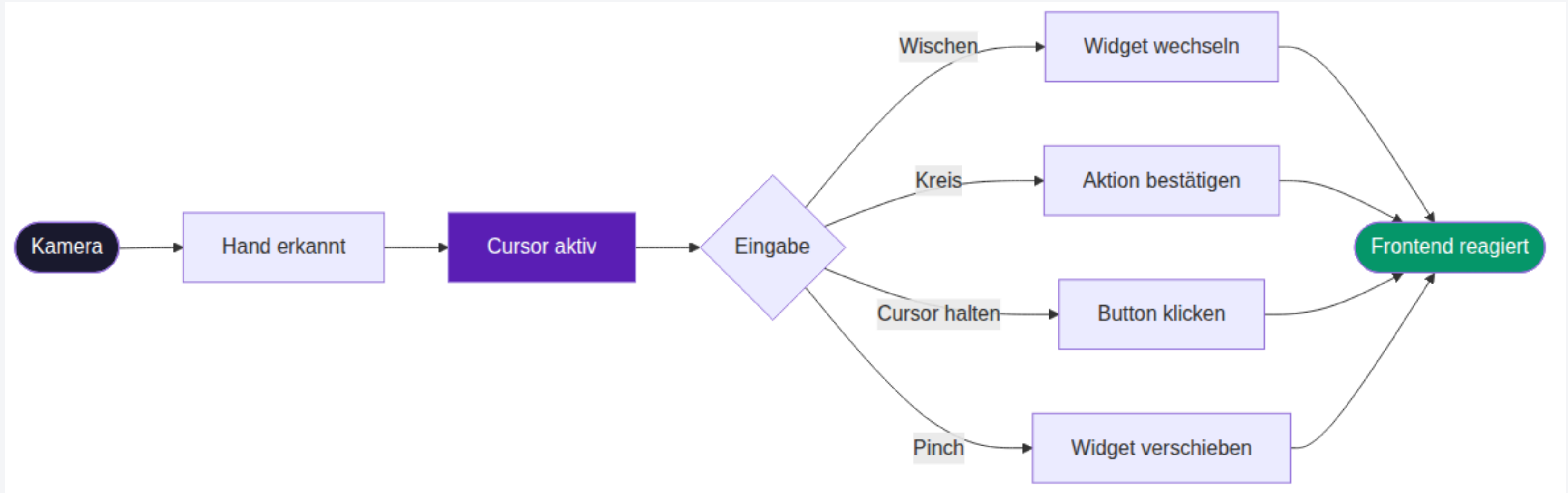
 Kreis — Edit-Modus beenden

Widget-Shop

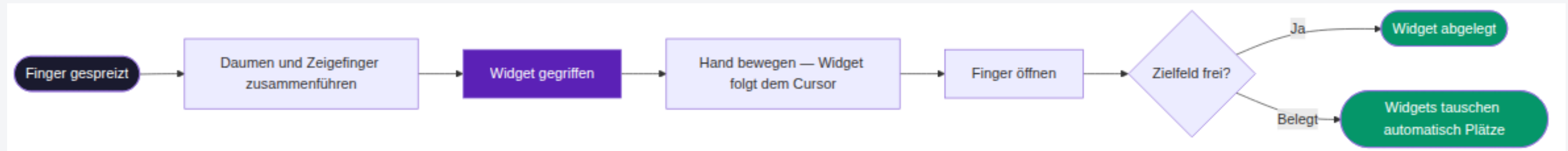
 Wischen — Widgets durchblättern

 Cursor halten — Widget auswählen & hinzufügen

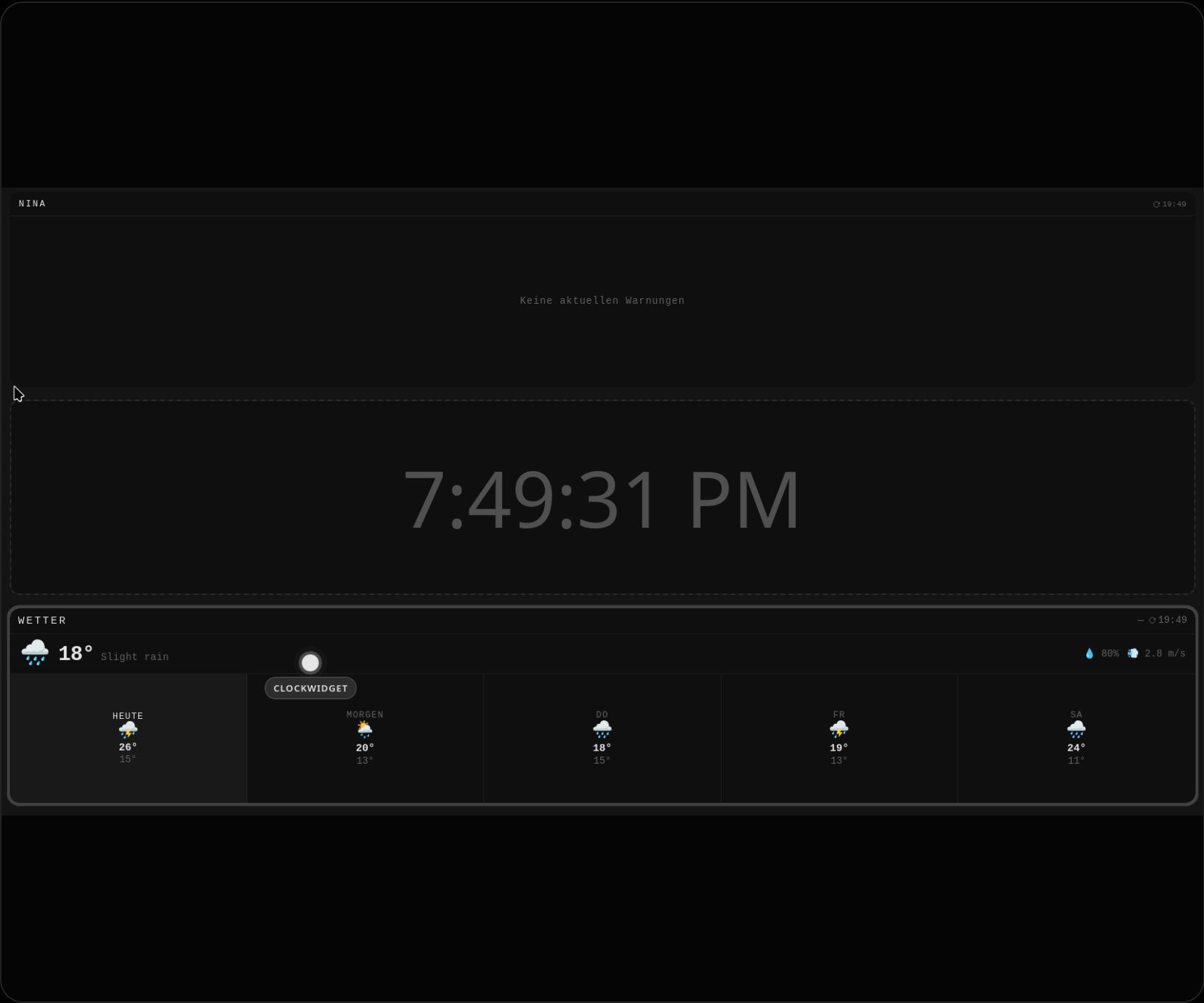
Gestensteuerung – Ablauf



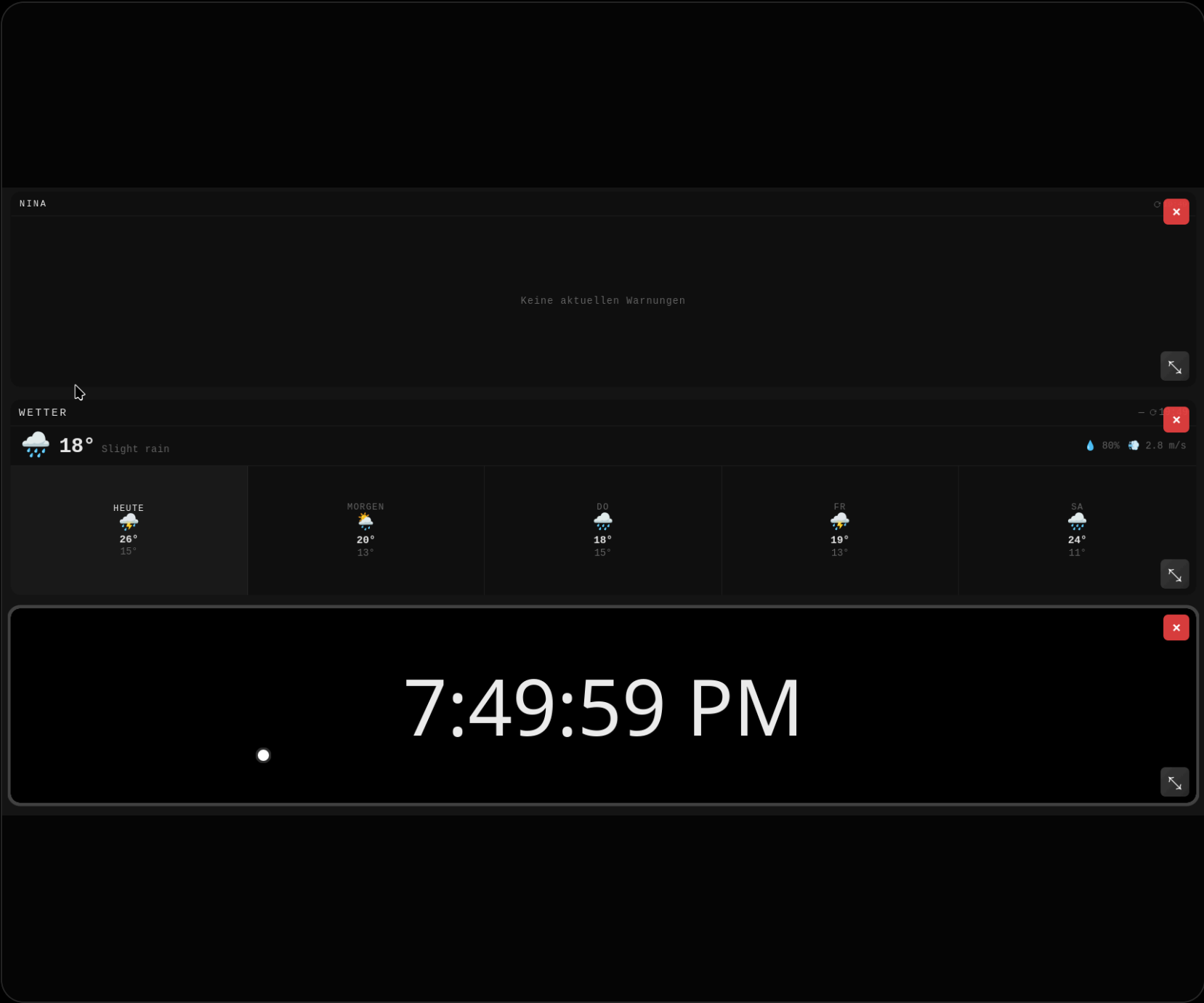
Pinch-to-Move — Schritt für Schritt



Pinch-to-Move — live in Aktion



Widget wird mit Pinch gegriffen und verschoben



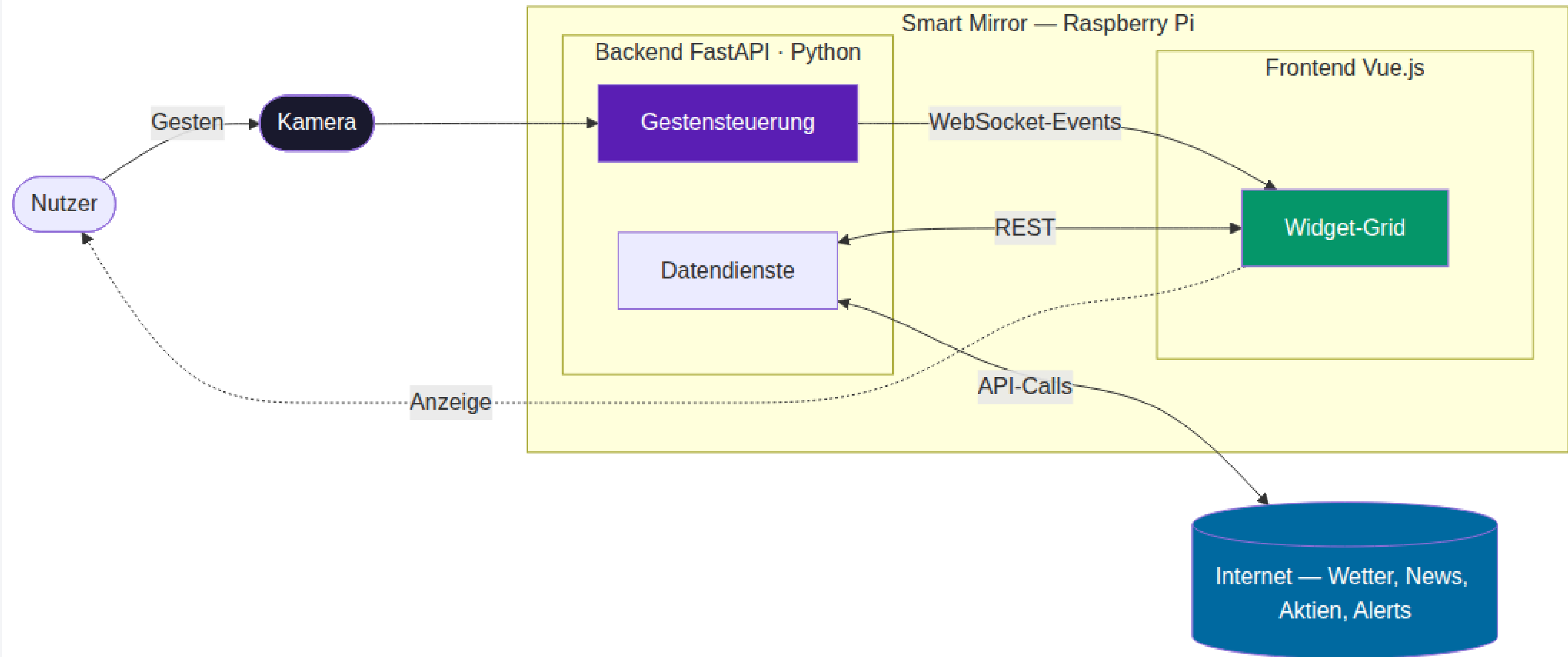
Clock- und Wetter-Widget haben automatisch die Plätze getauscht

03

Technische Umsetzung

Architektur, Design-Entscheidungen &
Implementierung

3.1 · System-Architektur — Überblick



3.1 · Zentrale Design-Entscheidungen

Modularer Monolith

Alle Backend-Services laufen **in einem Prozess** — keine Microservices.

- Kein Netzwerk-Overhead zwischen Services
- Ein einziges Deployment auf dem Raspberry Pi
- Klare Layer-Grenzen trotzdem vorhanden: `API → Services → Repositories → DB`
- Richtige Wahl für 3-köpfiges Team und diesen Projekt-Scope

Event-System via WebSocket

Ein einziger `/ws`-Endpunkt für alle Echtzeit-Events:

- Erkannte Geste → Frontend
- UI-Aktionen (Widget wechseln, greifen, ablegen)

Frontend: Composable & Registry

Jedes Widget ist eigenständig registriert — neue Widgets lassen sich **ohne Änderung am Core-System** hinzufügen.

Tech-Stack auf einen Blick

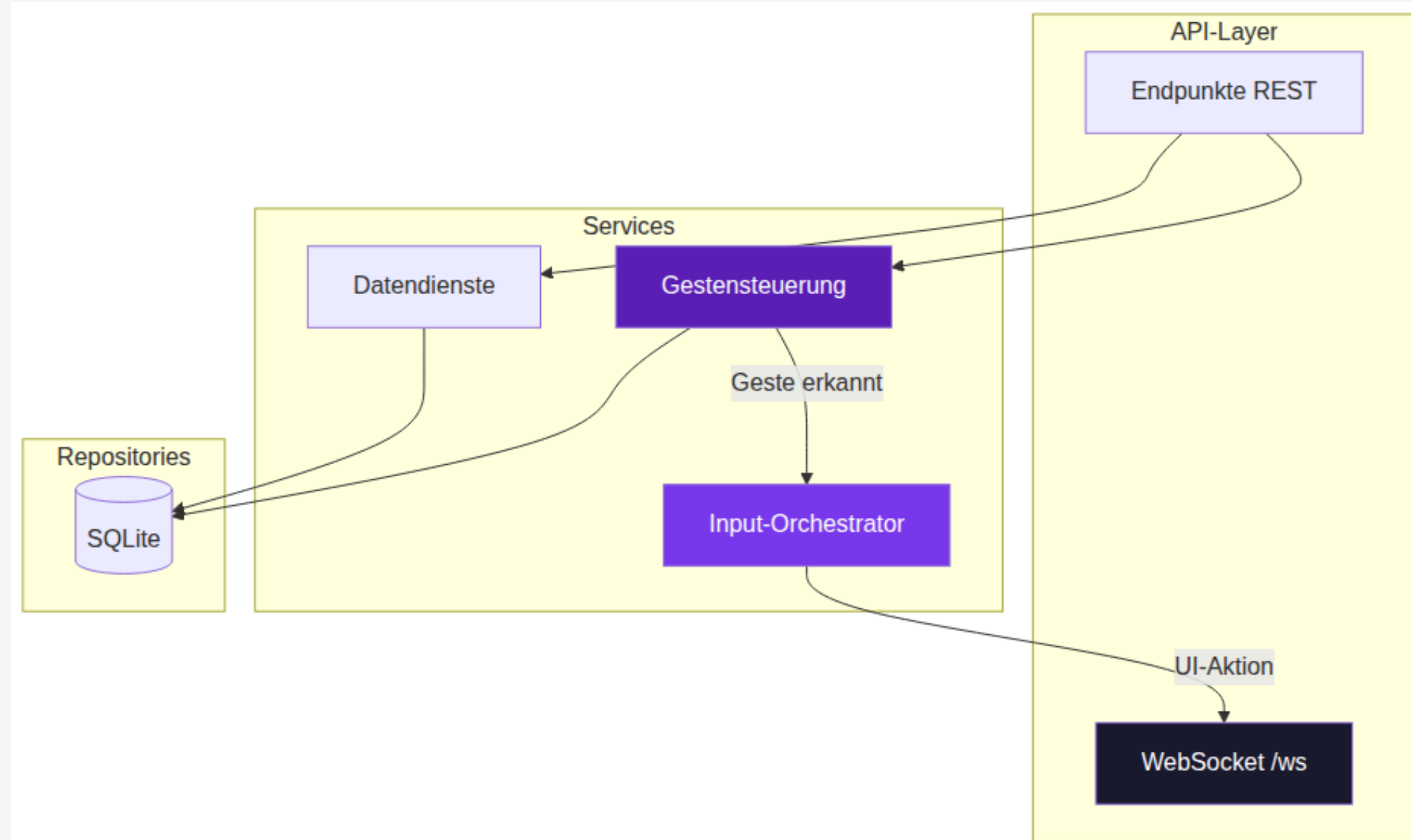
BEREICH	TECHNOLOGIE	EINSATZ
Backend-Framework	FastAPI · Python 3.12	REST-API + WebSocket
Hand-Tracking	MediaPipe (Google)	21 Landmarks pro Hand
Datenbank	SQLite	Konfiguration, Cache, Sessions
Konfiguration	Pydantic BaseSettings	150+ typsichere Parameter
Realtime	asyncio · WebSocket	Event-Broadcasting
Frontend	Vue.js · Vite · Tailwind CSS	Widget-Grid, Cursor-Overlay
Hardware-Plattform	Raspberry Pi	Kamera + Hosting

3.2

Backend

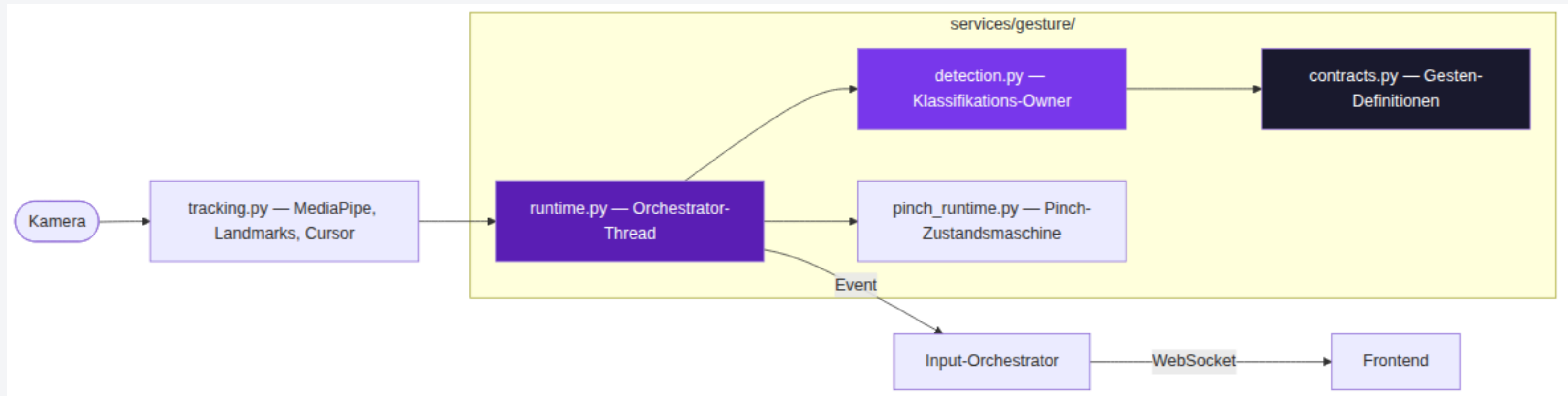
Architektur & Gestensubsystem

Backend – Schichtarchitektur



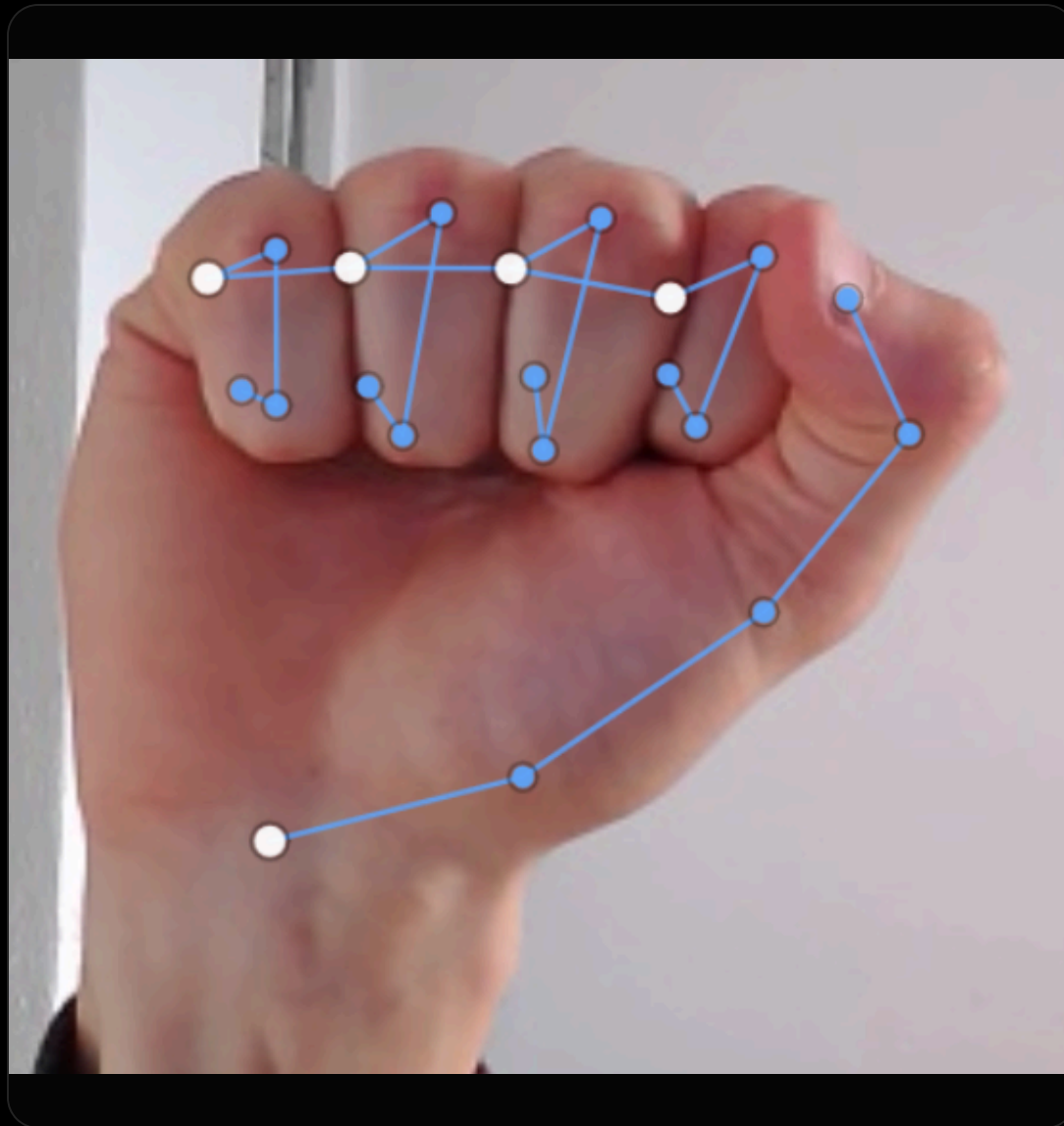
Datenfluss: Kamera → Gesture-Service → Input-Orchestrator → WebSocket → Frontend

Gestensubsystem — Aufbau & Verantwortlichkeiten

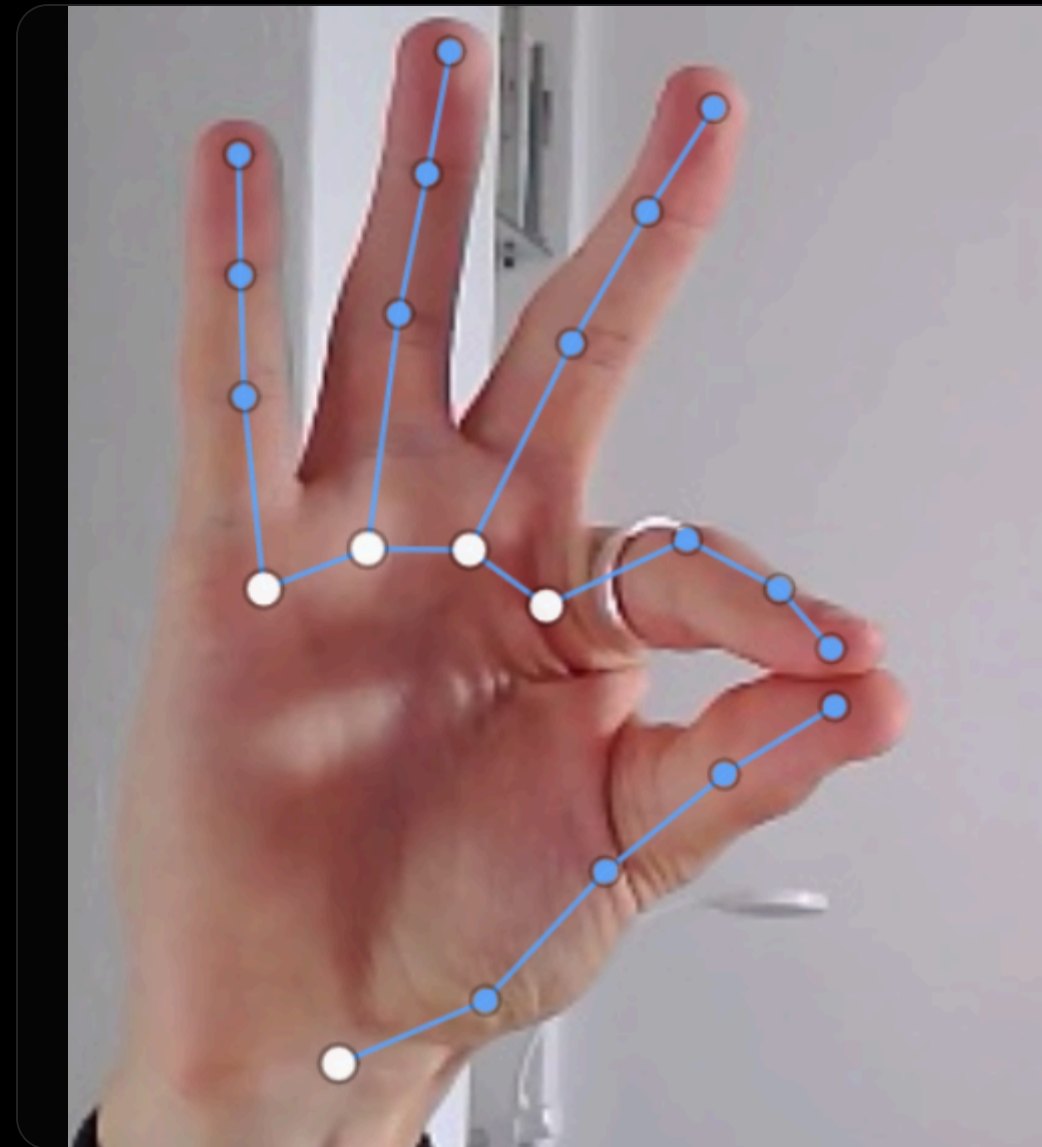


Klare Ownership: `detection.py` klassifiziert — `runtime.py` orchestriert nur. Keine gemischten Verantwortlichkeiten.

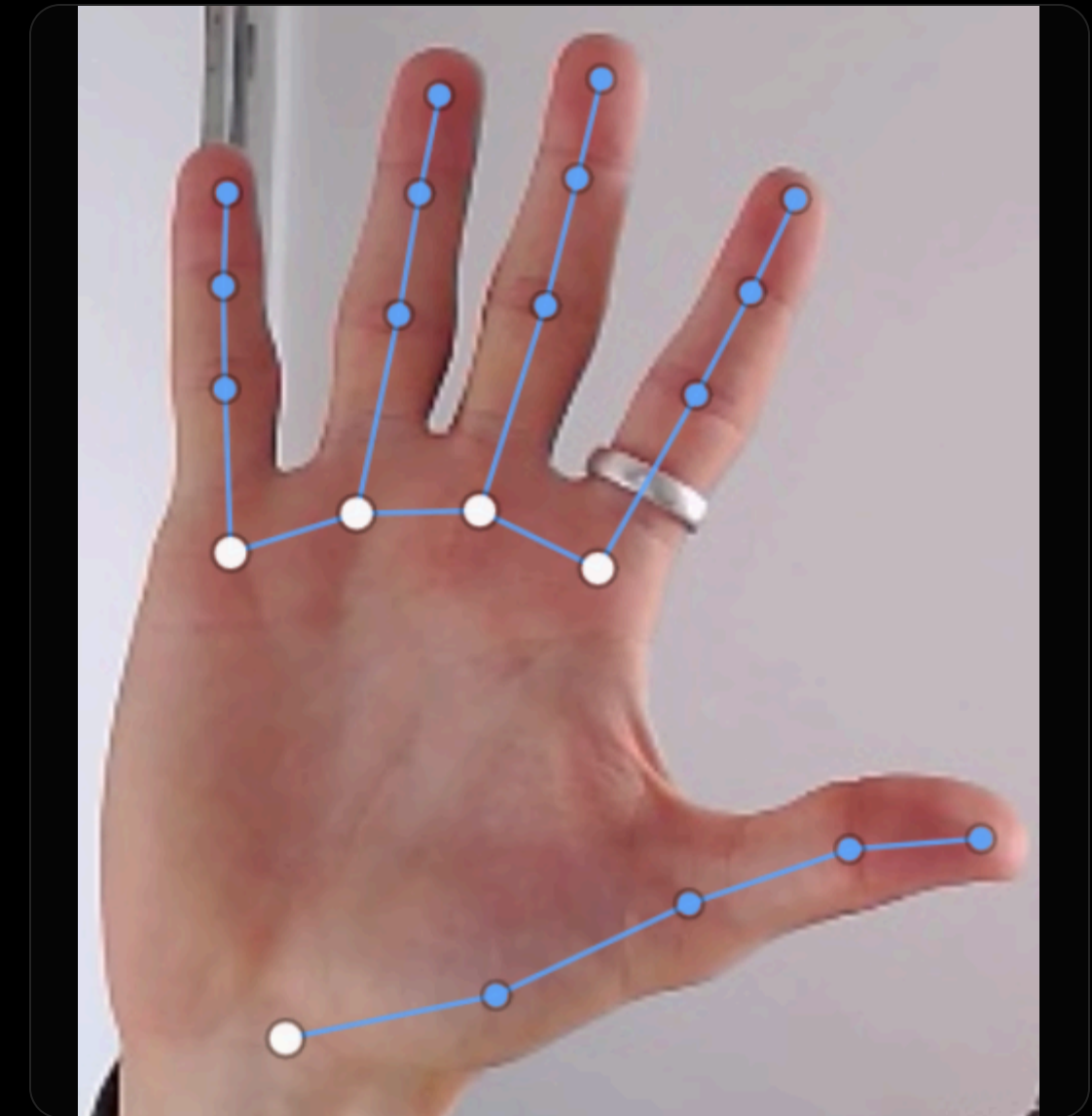
MediaPipe Hand-Tracking — so sieht das System die Hand



Faust — Startpose für die Kreis-Geste



Pinch — Daumen + Zeigefinger zusammen



Offene Handfläche — Startpose für Wischen

MediaPipe erkennt **21 Landmarks** pro Hand in Echtzeit — Basis für alle Gesten-Klassifikationen.

3.3 · Frontend-Architektur

Aufbau: Vue 3 + TypeScript

Klare Schichttrennung — jede Schicht hat eine Verantwortung:

- **Komponenten** — Darstellung (UI)
- **Composables** (`use...`) — Geschäftslogik, wiederverwendbar, testbar
- **Services** — Datenzugriff & externe APIs

Grid & Layout: 4×4 Grid-System · Drag & Drop · variable Widget-Größen · automatische Layout-Speicherung

Composables — Logik aus der UI ausgelagert

Logik in eigenständigen `use...`-Dateien:

- `useWidgetManager` — Widget-Zustand & Positionen
- `useHandTracking` — Gesten-Events vom Backend
- `useModuleShop` — Widget-Bibliothek

Vorteil: Schlanke Komponenten · Zustände zentral verwaltet · einfacher zu testen

3.3 · Das Widget-System — Erweiterbarkeit by Design

Registry Pattern + Factory

```
// widgetRegistry.ts
const modules = import.meta.glob(
  '../components/widgets/*.vue',
  { eager: true }
)
```

Widgets werden **automatisch erkannt** — kein manuelles Eintragen nötig. `getWidgetComponent(name)` lädt die passende Komponente dynamisch.

Singleton-Zustand in `useWidgetManager` stellt sicher, dass alle Komponenten dasselbe Widget-Layout sehen.

Neues Widget in 3 Schritten

1. Neue `.vue`-Datei anlegen
2. Template + Logik + Styling implementieren
3. Im Widget-Ordner speichern

Fertig. Das System erkennt und registriert das Widget automatisch — ohne Änderung am Core.

Der Smart Mirror ist dadurch flexibel erweiterbar: jede neue Funktion ist ein eigenständiges Modul.

Das Widget-System — Spotify-Widget

TECHNISCHER ABLAUF

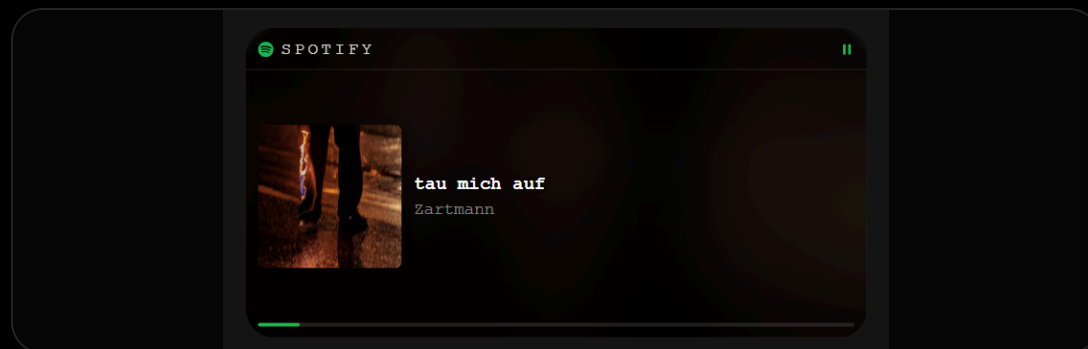
01 Authentifizierung (OAuth 2.0)

- Einmalige Anmeldung über Spotify
- Backend speichert Access- & Refresh-Token in SQLite
- Token wird automatisch erneuert, bevor er abläuft

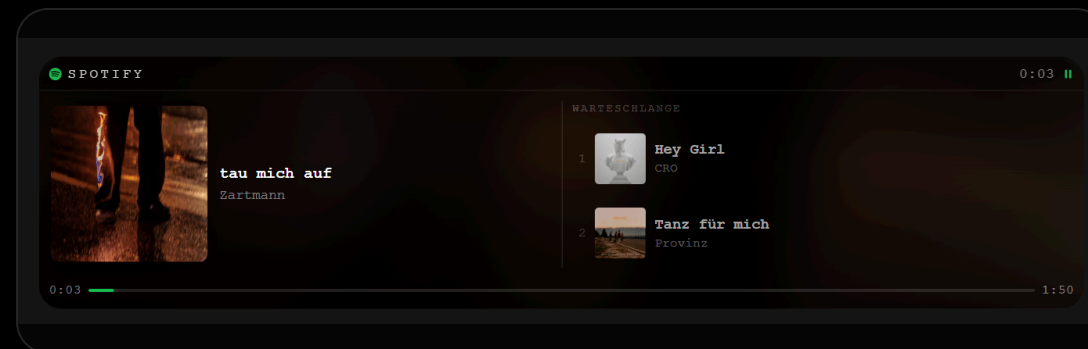
02 Datenabruf

- Frontend fragt alle 5 Sekunden das Backend an
- Backend ruft parallel die Spotify API ab:
 - Aktueller Song + Fortschritt
 - Nächste 3 Songs in der Warteschlange
- Fortschrittsbalken läuft clientseitig sekundlich weiter (kein Over-Polling)

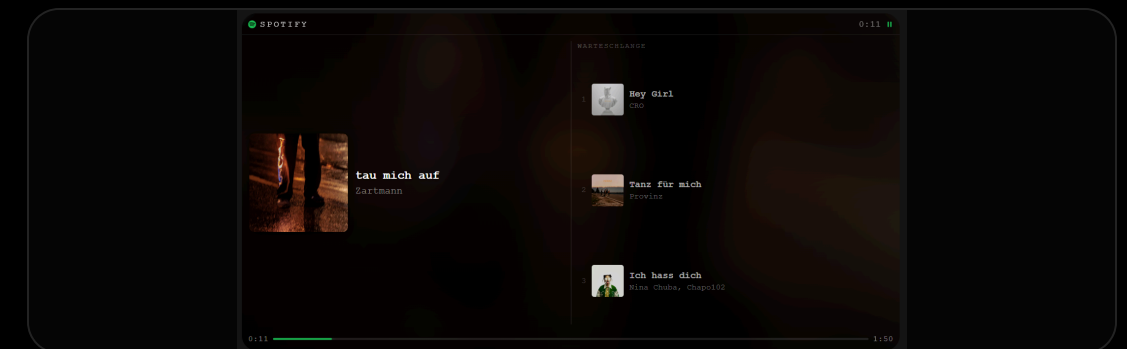
03 Anzeige



Klein

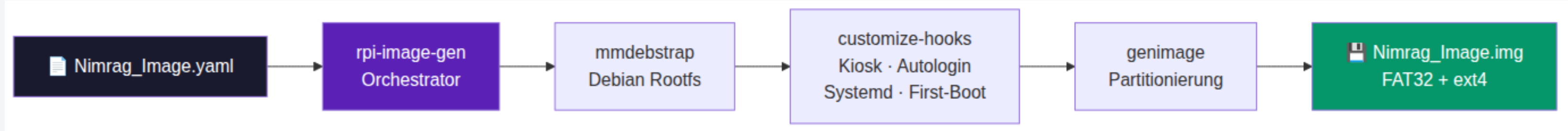


Mittel



Groß

4 · Hardware — Image-Build-Pipeline



Vom Code zum fertigen Spiegel-Image

Eine einzige YAML-Datei treibt den gesamten Build und erzeugt ein einzelnes `.img`-File — direkt auf SD-Karte flashbar.

First-Boot-Setup führt beim ersten Hochfahren aus:

- `npm install` + `pip install`
- Datenbankmigrationen
- Systemd-Services aktivieren

Hardware-Komponenten

- Raspberry Pi (Herzstück)
- Zwei-Wege-Spiegel + Display + Rahmen
- USB-Kamera für Gestensteuerung
- LED-Strip (TP-Link Tapo + MOSFET)

5 · Qualität — Tests & CI/CD

Automatisierte Tests

- **pytest** mit Coverage-Report (`coverage.xml`)
- Backend-Tests: Endpunkte, Services, Repositories
- Linting: **black** (Formatierung) + **flake8** (Stil & Fehler)
- Alle Tests laufen isoliert mit Test-SQLite-Datenbank

CI/CD Pipeline (GitHub Actions)

Push / Pull Request



- ① Lint & Format-Check
- ② Backend Tests + Coverage
- ③ Frontend Build
- ④ Quality Gates
 - ↓ (nur main-Branch)
- ⑤ Deploy-Artefakt

Pipeline blockiert Merges bei fehlgeschlagenen Tests oder Lint-Fehlern — Qualitätssicherung ist verpflichtend, nicht optional.

6 • Projektmanagement

Vorgehen

- Agile Entwicklung in Sprints
- Feature-Branches + Pull Requests
- Code Reviews vor Merges in `dev` und `main`
- `main` = stabile Version, `dev` = Integration

Learnings

01
Früh entscheiden. Architektur & Design tragen das ganze Projekt.

02
Testen zahlt sich aus. Fehler fallen früh auf, bevor sie sich ausbreiten.

03
Früher anfangen. Puffer einplanen — der Endspurt kommt schneller als gedacht.

04
Kleine Pull Requests. Schnellerer Review, weniger Merge-Konflikte.

Aufgabenverteilung

BEREICH	VERANTWORTUNG
Backend & Gestensteuerung	Louis
Frontend & Widgets	Sebastian
Hardware & Spotify-Widget	Jannik
Gemeinsam	Architektur, Reviews

7 • Ausblick & nächste Schritte

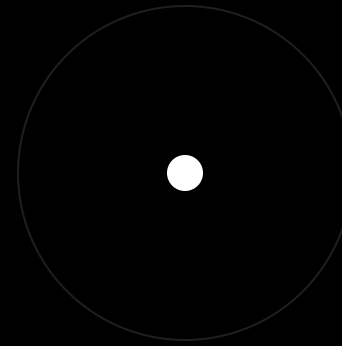
Offen / In Entwicklung

- Sprachsteuerung (Deutsch) — Integration fast abgeschlossen
- Kalibrierung der Gesten per Nutzer-Profil
- Deployment-Pipeline für Raspberry Pi finalisieren
- LED-Steuerung via Smart-Home-Integration

Mögliche Erweiterungen

- Weitere Widget-Typen (Kalender, Smart-Home)
- Nutzerprofile & Personalisierung
- Verbesserte Gestengenauigkeit via DTW-Kalibrierung
- Mobile Companion-App

Jetzt zeigen wir das System live — alle Gesten in Aktion.



Live-Demo

Nimrag in Aktion

Swipe

Circle

Pinch-to-Move

Cursor-Hold

Danke für eure Aufmerksamkeit.

Fragen?

Louis Reißer

Backend · Gestensteuerung

Sebastian Rieger

Frontend · Widgets

Jannik Haase

Hardware · Spotify-Widget